DREW

Lecture 18: LC-3 Programing

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

10/11/2024

DREW

3 Control Instructions

Branchings
Jumps
TRAPs

Branch
PC-relative

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	N	Z	P	PCoffset9								

- **Opcode**: 0000 **4 operands**: three single-bit CC checker and a 9-bit PC offset. **Function**: Upon match with current CC, go to location.

- **Recall**: NOT, AND, ADD, LD, LDI, LDR, LEA all *sets CC*.
- Based on the result, exactly one flag is set to 1.

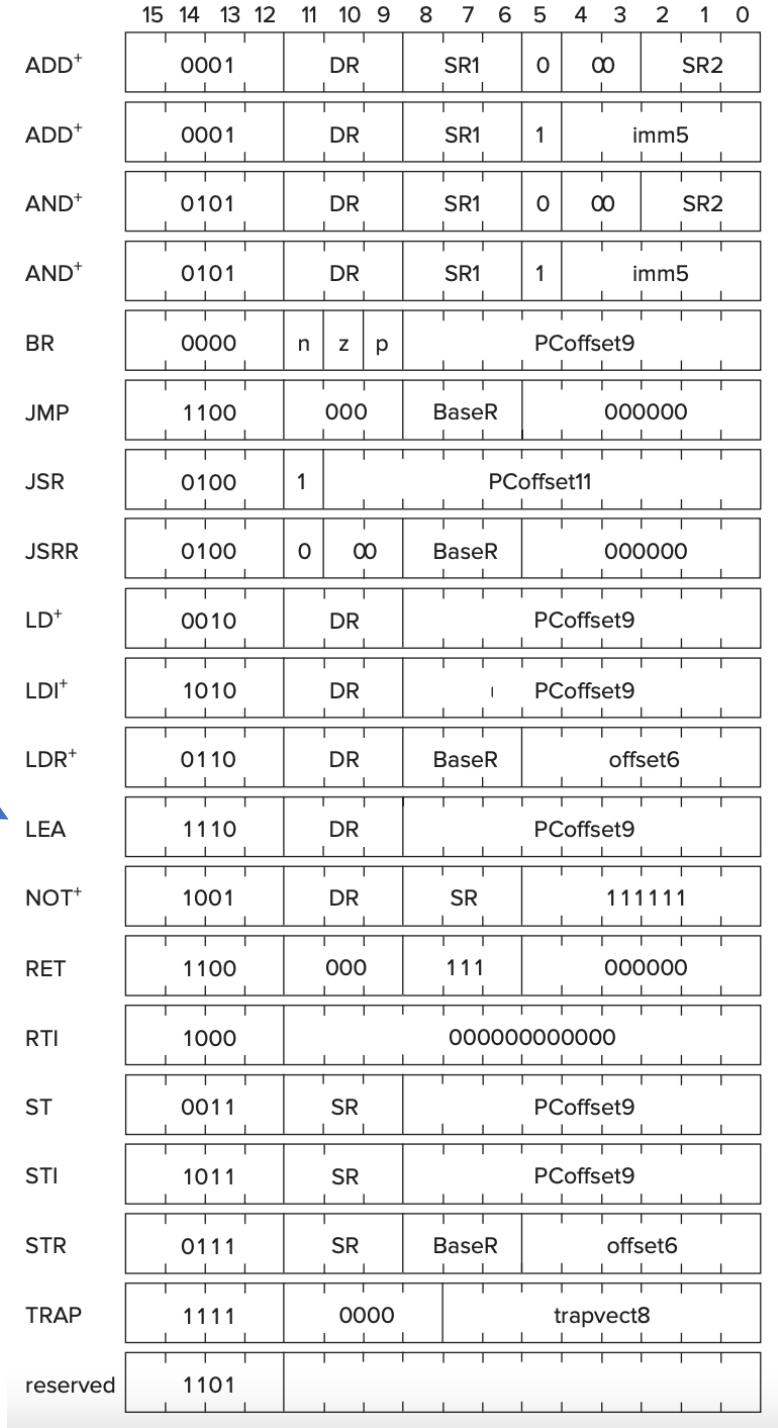
- BR uses CC checker to match with the current CC.
  - BRn, BRnz, BRnp, BRnzp, BRz, BRzp, BRp
- If matched, go to PC+PCoffset9 location.

- **Ex.**: if-statement, looping-condition, subroutine calls, etc.

Instructions

- Instructions that set the CC:
  - NOT, AND, ADD
  - LD/LDI/LDR/LEA

typo



	15 14 13 12	11 10 9	8 7 6	5	4 3	2 1 0
ADD+	0001	DR	SR1	0	00	SR2
ADD+	0001	DR	SR1	1	imm5	
AND+	0101	DR	SR1	0	00	SR2
AND+	0101	DR	SR1	1	imm5	
BR	0000	n	z	p	PCoffset9	
JMP	1100	000	BaseR	000000		
JSR	0100	1	PCoffset11			
JSRR	0100	0	00	BaseR	000000	
LD+	0010	DR	PCoffset9			
LDI+	1010	DR	PCoffset9			
LDR+	0110	DR	BaseR	offset6		
LEA	1110	DR	PCoffset9			
NOT+	1001	DR	SR	111111		
RET	1100	000	111	000000		
RTI	1000	000000000000				
ST	0011	SR	PCoffset9			
STI	1011	SR	PCoffset9			
STR	0111	SR	BaseR	offset6		
TRAP	1111	0000	trapvect8			
reserved	1101					

DREW

Examples

https://wchargin.com/lc3web/

- Set PC = x30F6.

Now let's focus on the CC.

Address	Instruction	Ins. In Hex	Result	NZP
x30F6	LEA R1, #-3	xE3FD	x30F4 in R1	001
x30F7	ADD R2, R1, #14	x146E	x3102 in R2	001
x30F8	ST R2, #-5	x35FB	N/A	
x30F9	AND R2, R2, #0	x54A0	x0000 in R2	010
x30FA	ADD R2, R2, #5	x14A5	x005 in R2	001
x30FB	STR R2, R1, #14	x744E	N/A	
x30FC	LDI R3, #-9	xA7F7	x0005 in R3	001

Examples

- Given R0 = x0003 and R1 = x0000

Instruction	**NZP**	**BR flag(s) satisfied**
ADD R1, R1, R0	001	p, np, zp, nzp
AND R1, R1, #0	010	z, nz, zp, nzp
ADD R1, R1, #-3	100	n, nz, np, nzp

- BRnzp means branching no matter what.

Jump Register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	Base			0	0	0	0	0	0

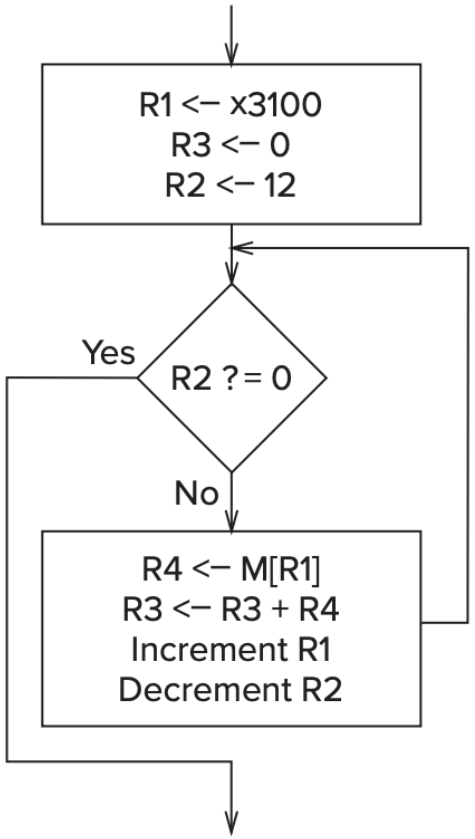
- **Opcode**: 1100 **1 operand**: a 3-bit indicating R0~R7 whose content is the base. **Function**: Unconditionally branch to location.

1. Assign content in the base register to PC.

- Same functionality as BRnzp.
- However, BR uses PCoffset. What if the destination is too far away?
- JMP can go to all the memory locations.

Ex.: Adding up 12 Integers

- Say program starts at x3000 and numbers at x3100.
- Initialization: Put the address of 1st number in R1. Use R3 for the running sum and R2 as a loop counter.



1. Is R2 zero; i.e., already added up all num.s?

If yes, terminate; otherwise, gets into loop.

[The loop]

1. Load R4 with current number, addressed by R1.
2. Update R3 with R3+R4.
3. Update both address of next number and the counter.
4. Go back to loop condition.

Ex.: Adding up 12 Integers

Address	Instruction	Note	Ins. in Hex
x3000	LEA R1, x00FF	x3001+x00FF=x3100	xE2FF
[Set R2, R3 to zeros; HW3]			
x3001	ADD R2, R2, #12	R2 ← R2+12	x14AC
x3002	BRz #5	Check if R2 is zero;	x0405
[loop starts here]		if so, goto x3008.	
x3003	LDR R4, R1, #0	R4 ← Mem[R1]	x6840
x3004	ADD R3, R3, R4	R3 ← R3+R4	x16C4
x3005	ADD R1, R1, #1	R1 ← R1+1	x1261
x3006	ADD R2, R2, #-1	R2 ← R1-1	x14BF
x3007	BRnzp #-6 /	Goto x3002.	x0FFA

Ex.: Adding up 12 Integers

Loading bin/hex instructions to LC-3: Use the "Raw" button.

- Can either type/paste in the ins., or simply drop a file containing the ins.

- Then, click "Process". If no error, can "Load into Simulator".
- **Program**:

```
3000
E2FF
14AC
0405
6840
16C4
1261
14BF
0FFA
F025
```

Data:

```
3100
0001
0002
0003
0004
0005
0006
0007
...
000C
```

Will come back to errors.

Can then "Finish" program and R3=x004E.

DREW

TRAP

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	TRAPvect8							

- **Opcode**: 1111 **1 operand**: an 8-bit vector for specifying the TRAP type.
  Function: Calls a *Service Routine* and return to the PC location.
- TRAPvect8: x23: Input a char. from keyboard to R0.
  x21: Output a char. in R0 to the monitor.
  x25: Halt the program.
- **Ex.**: Set R0=x0041, Mem[x3000]=xF021, should display "A".
  Set Mem[x3001]=xF023, input "B", R0 ← x0042
- **Ex.**: HALT as the last instruction in last example.
- Note that, HALT overwrites registers.

Questions?

- **Short Summary**
  - All the instructions with the 0's,1's are part of the *Machine Language*, what the computer really reads.
  - All programming languages, are ultimately translated to machine language before execution.

Programming

- To solve a problem in the machine instruction level, need to break down the problem.

Systematic Decomposition: a top-down approach to decompose the problem into simpler tasks, and those into even simpler ones, etc.,

- until every task can be implemented using instructions.
  - Essentially, 3 ways to break it down.
  - Then, perform *Stepwise Refinement*.

Then, need to re-construct the solution by combining/organizing those of the subtasks.

Programming

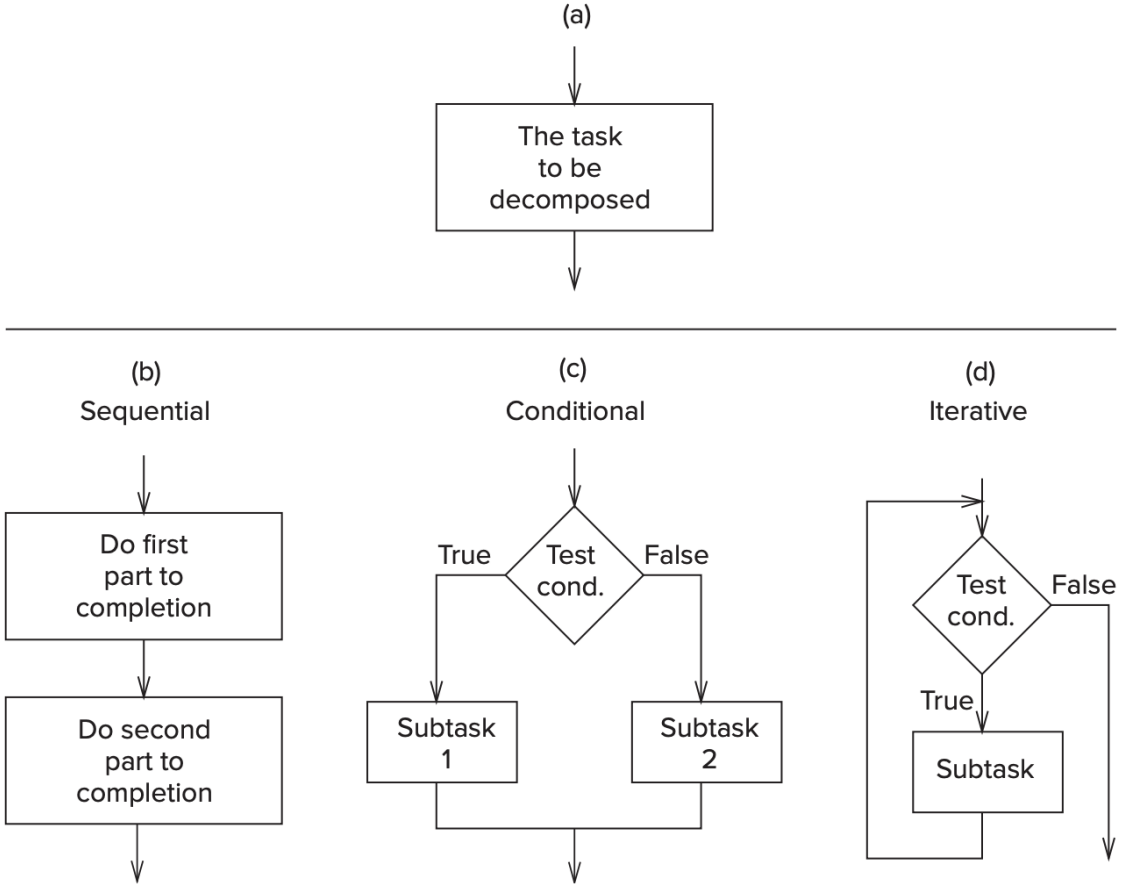
3 ways to break it down

- *Sequential construct*
  - **Ex.**: Read from keyboard → add

    input to a number → Output to console.

- *Conditional construct*
  - **Ex.**: If CC is P, then subtract 1 from R1;

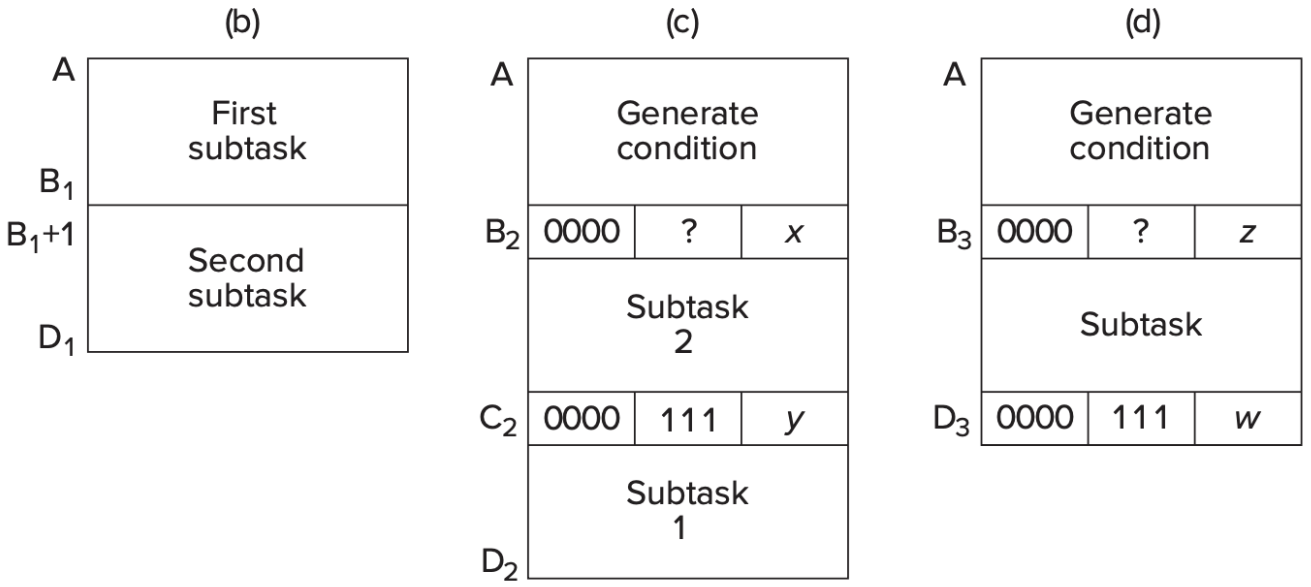
    otherwise, add 1 to R2.



- Iterative construct
  - **Ex.**: More numbers to add to the running sum?

    If so, get the next and add and check again; otherwise, break from loop.

Programming



3 ways to break it down

- *Sequential construct*
  - two subtasks in linear fashion

- *Conditional construct*
  - BR into Subtask 1 if condition (B_2) satisfied;

  otherwise, Subtask 2 is executed; after Subtask 2 is done, BR away (C_2) unconditionally, since Subtask 1 should not be reached.

- *Iterative construct*
  - BR away if condition (B_3) satisfied; otherwise, gets into loop; after one iteration is done BR (D_3) back to either A or B_3 to test again.

Programming

Stepwise Refinement: Essentially, decide for each subtask, what procedure to take.

- Do we need to break down Subtask 1? What construct should it be?
- Same for Subtasks 2,3, Sub-subtasks 1,2,3, etc.

Lastly, convert to instructions with *Comments* so both you and I know what you are doing.

- Containing sufficient amount of comment is required for all programs, in any course/job.
- In LC-3, ; is used for commenting.
- So, anything goes to the right are ignored by the simulator.

Programming

Lastly, convert to instructions.

- Note that, LC-3 uses PC as reference quite often.
- As a result, the PCoffsets are not decided before every instruction is written in correct places.
- So, need to leave some blank while figuring out the whole program.

- Now, the programming part is done. Let's test it out.
- What if … errors/unexpected results?
- The simulator tells you Sytax errors (put R8 somewhere).

Debugging

For logical/data errors, need to debug.

- LC-3 provides ability to setup *breakpoint*:
  - Click the dropdown arrow on the line you wish to put a breakpoint upon, "set breakpoint here"/"clear this breakpoint."
  - Trace through registers and memory locations for data storage.

- Can "Step" which gets into subroutines; or "Next" which stays in current routine by fully executing subroutines. (Step-in and Step-over in Eclipse)

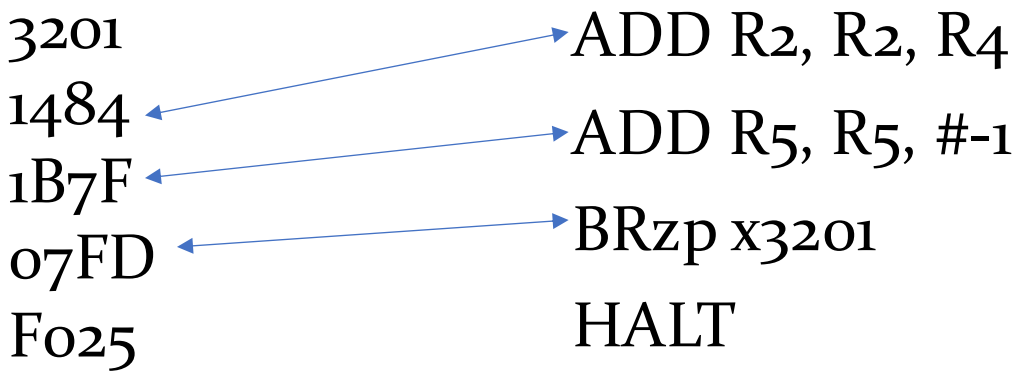
- Can "Finish" which fully execute the current subroutine, but will stop upon breakpoint. Can "Run", until a breakpoint/HALT is encountered.

- Can also use trap to print to console.

Debugging - Multiplication

Ex.: Trying to multiply two integers in R4 and R5.

- **Problematic Program**:



- set R4 = x000A (#10) and R5 = x0003, then result is R2 = x0028 (#40!).

Debugging - Multiplication

- What to look for when debugging

```
x3201  ADD R2, R2, R4
x3202  ADD R5, R5, #-1
x3203  BRzp x3201
x3204  HALT
```

By single-stepping

(a)

PC	R2	R4	R5
x3201	0	10	3
x3202	10	10	3
x3203	10	10	2
x3201	10	10	2
x3202	20	10	2
x3203	20	10	1
x3201	20	10	1
x3202	30	10	1
x3203	30	10	0
x3201	30	10	0
x3202	40	10	0
x3203	40	10	−1
x3204	40	10	−1
	40	10	−1

(b)

PC	R2	R4	R5
x3203	10	10	2
x3203	20	10	1
x3203	30	10	0
x3203	40	10	−1

By setting breakpoint at BR

Should've stopped there.

Loop executed 1 more time than desired.
Change to just BRp.

DREW

Questions?

- More debugging examples on textbook you can read and practice to get familiar with the simulator interface.

- **Next Lecture**:
  - Assembly Language Programming

- **HW3** due tonight by 11:59pm
- **HW4** available tonight